A.1 大量平行工作如何對應到 GPU

GPU 使用大量執行單元與高記憶體頻寬處理許多相似工作；CPU 則通常在複雜控制與單一工作的低延遲上投入更多資源。這並不代表每個 GPU 執行緒都比 CPU 執行緒快：資料量太小時，啟動與傳輸成本可能抵消計算收益。

圖形處理與通用運算共用可程式化處理器；圖形管線還有光柵化、紋理取樣與輸出合成等專用功能。應用程式看到的管線階段，不一定各有一批專屬的實體 SM 核心。

本附錄以 CUDA 與 Tesla 的歷史架構建立背景；32-byte sector、32-bank 和 occupancy 算例則各自明列抽象假設，不能合併當成 GeForce 8800 的硬體規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

圖形管線與插值

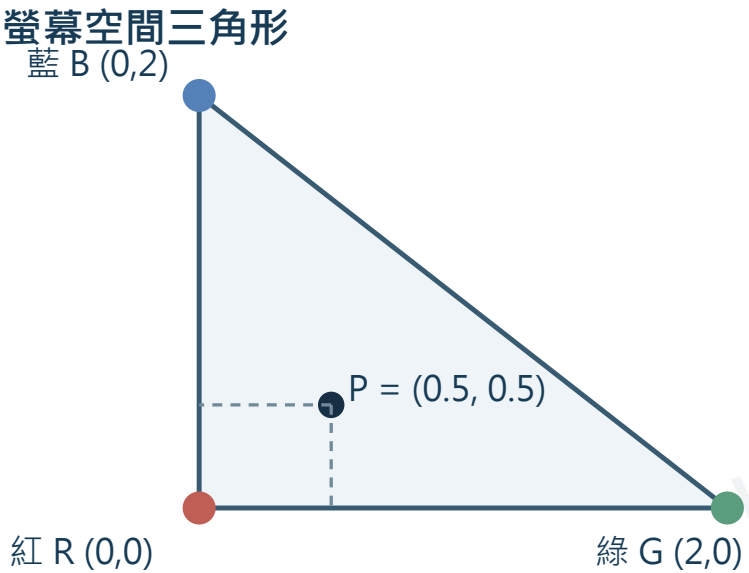
頂點著色器產生頂點位置與屬性；三角形經裁切與光柵化後形成片段（fragment），再由片段著色器計算顏色，並依深度、模板與混色規則更新畫面。片段可能因深度測試失敗而不成為最終可見的像素。

螢幕上的三角形頂點為 (0,0)、(2,0)、(0,2)，且三個頂點的 clip-space w 都是 1。點 (0.5,0.5) 的重心權重為 (1/2,1/4,1/4)；若三個頂點分別是純紅、綠、藍，插值結果為 (0.5,0.25,0.25)。

若頂點的 w 不同，須先以 $(\lambda_i/w_i)/\sum_j(\lambda_j/w_j)$ 計算透視校正後的屬性權重。紋理濾波可能讀取多個鄰近 texels；輸出合成也可能讀取既有的顏色或深度，因此流量不只有最後一次寫入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用頂點權重計算插值



P = ½R + ¼G + ¼B

權重和 = 1，均非負 → P 在三角形內

假設各頂點 clip w = 1

插值色 (R,G,B) = (0.5, 0.25, 0.25)

點的座標滿足 $P = \frac{1}{2}(0,0) + \frac{1}{4}(2,0) + \frac{1}{4}(0,2) = (0.5, 0.5)$。權重和為 1 且皆非負，因此 P 位於三角形內。三個頂點的純紅、綠、藍屬性以相同權重相加，得到 $(0.5, 0.25, 0.25)$。此例三個 clip w 相同。若沿用螢幕權重而改取 $w = (1, 2, 4)$，校正前權重為 $(1/2, 1/8, 1/16)$，總和 $11/16$；正規化後為 $(8/11, 2/11, 1/11)$。以原螢幕權重直接插值，會得到不同結果。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A.2 主機、裝置與端到端成本

獨立 GPU 透過 I/O 連線與主機相接，並有自己的裝置記憶體；驅動程式管理命令、映射與完成通知。整合式 GPU 或共享記憶體設計可能減少複製，但仍要考慮資料可見性與共享頻寬。統一位址不代表資料移動完全沒有成本。

假設各階段依序執行，資料複製與計算不重疊：輸入和輸出各 64 MiB，連線每方向有效頻寬為 8 GB/s，啟動需 20 µs，kernel 執行需 2 ms。兩次傳輸合計

$$2 \times 64 \times 2^{20} / (8 \times 10^{9}) = 16.777216 \text{ ms}.$$

加上啟動與計算，總時間為 18.797216 ms。若 CPU 完成同一工作需 25 ms，端到端加速比約為 1.330；只比較 25/2=12.5 會漏掉傳輸與啟動。若資料留在裝置上連續執行 10 次相同 kernel，只需一次輸入與輸出，總時間為 36.977216 ms；能否這樣攤提取決於實際資料依賴與使用情境。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A.3 Grid、block 與 thread 的索引

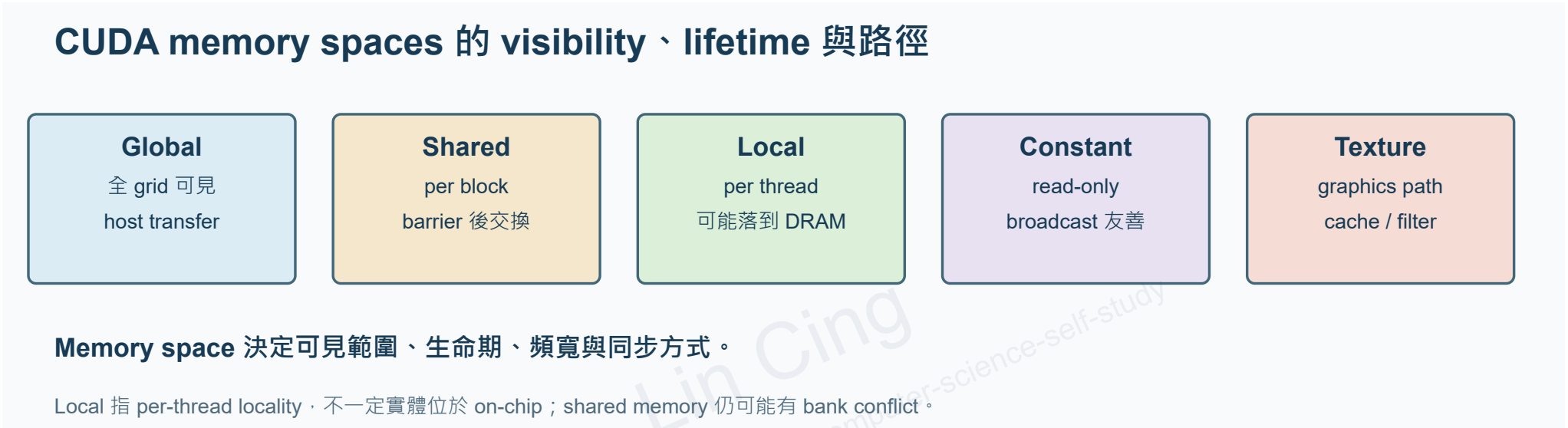
一維kernel以 $i = blockIdx.x \cdot blockDim.x + threadIdx.x$ 對應輸出。N=1000、block=256需ceil(1000/256)=4 blocks；前三個處理0–767，最後block的threads0–231處理768–999，其餘24個不得訪問越界資料。

```
i = block_id * B + thread_id
if i < N:
    y[i] = f(x[i])
```

每個有效i唯一對應(⌊i/B⌋, i mod B)，所以覆蓋所有輸出且不重複寫。這段可直接跳過無效threads，因為沒有block barrier。若後續有共同barrier，不能照抄early return，必須讓需參與的threads一致到達。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Memory space 的 visibility 與 lifetime



Registers與local memory都是per-thread private，但local通常位於device DRAM，例如register spill或可動態索引的private array。Shared memory是per-block scratchpad，block結束後生命期結束。Global memory可跨blocks使用，但資料可見性需同步規則。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Memory space 的 visibility 與 lifetime （續）

Constant memory是device程式讀取的常數空間；同一warp讀相同位址可利用broadcast。Texture path提供特定格式、filtering與cache語意，不等於任意load都自動有相同性能。空間名稱描述程式模型，不單獨保證實體延遲。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Barrier 不只是等待計時器

Block barrier要求參與群組按規格到達，並建立對應記憶體操作的可見性。它不能替代跨block的全域同步，也不能假設「warp內看起來同時執行」就不需規格要求的同步。

對N不是block size倍數的tile，正確形式是所有threads先以predicate載入有效值或identity，全部到barrier，只有有效輸出thread計算，然後再次全部到barrier。第二次barrier防止快thread覆寫shared tile，而慢thread仍在讀上一輪。

不同blocks的執行順序與同時resident數不保證。若resident block spin等尚未排入的block，可能占滿所有資源而永遠無法前進；多kernel phases用前一kernel完成作全域邊界，比無條件跨block忙等更容易建立正確性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.4 SM、warp 與 scoreboard

32 個 lanes：兩條互斥分支的執行遮罩

Path A：lane 0–4 活躍

0	1	2	3	4	×	×	×
×	×	×	×	×	×	×	×
×	×	×	×	×	×	×	×
×	×	×	×	×	×	×	×

Path B：lane 5–31 活躍

×	×	×	×	×	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

有色數字 = 活躍；× = 遮罩。兩路各執行一次後再會合。

若兩路指令數相同：有效 lane-slots = 5 + 27；總 slots = 32 × 2；利用率 = 50%。

Warp是共同issue的一組threads，threads保有自己的register與logical狀態。歷史Tesla例warp32 threads、8個SP cores，以4個core cycles服務一個warp的32 lanes；這是吞吐寬度，不代表所有指令的result latency都是4 cycles。

A.4 SM、warp 與 scoreboard （續）

Scoreboard追蹤operand相依，scheduler只issue已ready的warp。某warp等待memory或相依結果時，可選另一個ready warp。Resident warps只是已配置資源的集合；若全部都在等barrier或同一memory事件，occupancy高仍可能無指令可issue。

PTX是virtual ISA，還需對目標架構編譯/JIT成實際machine code。PTX一條指令不必等於一個cycle或一條相同長度的SASS，故性能分析需分清模型層級。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Latency hiding 的有限模型

假設每4 cycles可issue一個warp instruction，每個warp只有一條長度32 cycles的dependent chain，無其他瓶頸。Round-robin選8個ready warps時，同warp相隔8×4=32 cycles，剛好可等前一結果；只有4個warps則相隔16 cycles，還需等16 cycles。

若依賴延遲增加到400 cycles，這個「每warp僅一個outstanding依賴」模型需要100個warps才能完全遮蔽，可能超過硬體容量。增加ILP、同時發出多個獨立load、改善cache與memory並行度，會改變所需warps數；不能只靠把occupancy填滿保證無stall。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divergence 的加權 lane-slot 計算

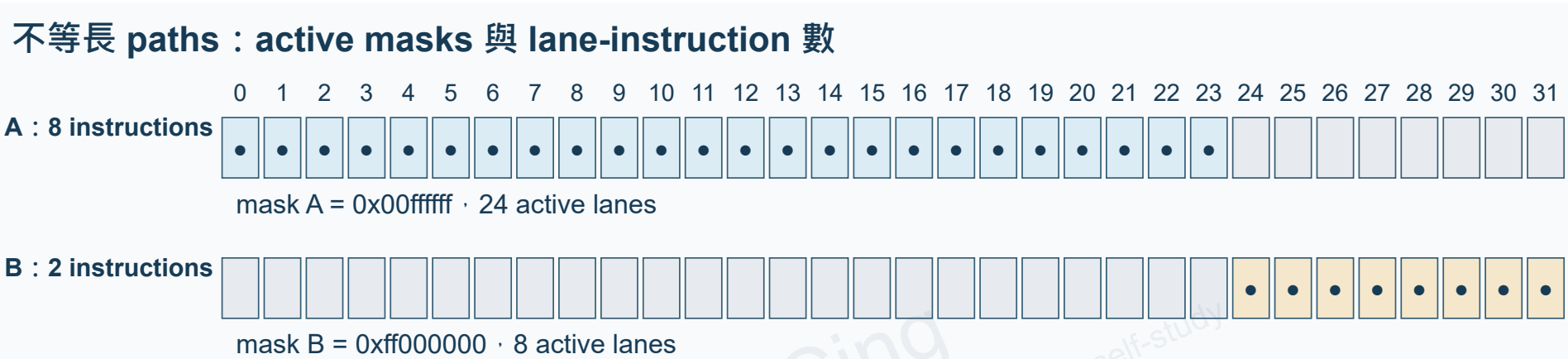
兩個互斥paths各有a、b個active lanes，a+b=W；path長度LA、LB指令。忽略branch與reconvergence額外成本，若兩條皆需執行，

$$\eta = \frac{aL_A + bL_B}{W(L_A + L_B)}.$$

只有LA=LB時才可簡化成1/2。取W=32、a=24、b=8、LA=8、LB=2，有效lane-instructions=24×8+8×2=208，issue capacity=32×10=320，所以η=65%。這是指令lane使用率，不直接等於整個kernel speedup或DRAM效率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divergence 的加權 lane-slot 計算 （續）



若某path完全無active lane，就不需執行該path，分母不能仍無條件加其指令數。Predication也需issue受predicate控制的指令，inactive memory操作是否形成transaction須依ISA語意，不能把false lane當成一定讀取DRAM。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Occupancy 是資源限制的整數問題

採抽象SM上限768 threads、24 warps、8 blocks、32768個32-bit registers、48 KiB shared memory，warp32。忽略allocation granularity。每block256 threads、每thread32 registers、每block16 KiB shared。

$$blocks \le \min\left(\left\lfloor \frac{768}{256} \right\rfloor, 8, \left\lfloor \frac{32768}{256 \times 32} \right\rfloor, \left\lfloor \frac{48}{16} \right\rfloor\right) = 3.$$

Active warps=3×8=24，occupancy100%。若block改128但shared仍16 KiB，shared只允許3 blocks，active warps12，occupancy50%。若shared也降8 KiB，可放6 blocks，回到24 warps。若原256-thread版register提高到64/thread，則register只允許2 blocks，occupancy16/24=2/3。

這些是resident上界，非實測利用率。Register減少若引起spill，可能增加DRAM流量，使較高occupancy反而更慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.5 Coalescing 的可重算交易模型

本節另採32-byte sector、32 lanes、每lane一個4-byte aligned word，只計一次warp request觸及的不同sectors，忽略cache命中與跨層合併；不是GeForce 8800的完整coalescing規則。

對byte address ai，需要sector集合 $\{\lfloor a_i/32 \rfloor\}$。連續words且base32-byte aligned時，128 bytes恰占4 sectors；base偏4 bytes時占5 sectors，讀取160 bytes但有用128，利用率80%。

Stride32 bytes或128 bytes時，每lane落在不同sector，都是32 sectors、1024 bytes，有用128 bytes，利用率12.5%；不同stride雖sector數相同，channel/row locality仍可不同。

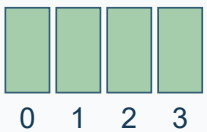
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.5 Coalescing 的可重算交易模型 （續）

Global request：同為128有用bytes，分別需要4或32個sectors

連續4-byte words

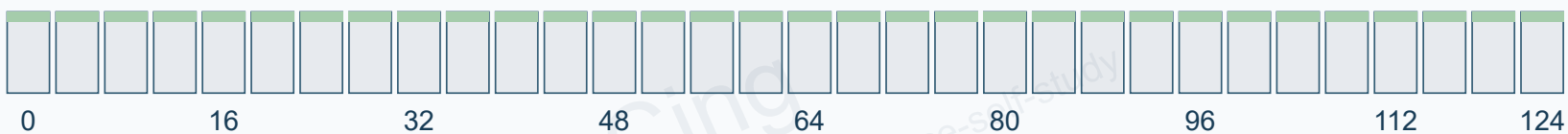
sector IDs 0–3



4 × 32 B = 128 B；100% 有用

Stride 128 bytes

sector IDs = 4i



每格都是32 B；綠色為有用payload。下列32 sectors共1024 B，僅128 B有用，利用率12.5%。

若全部lanes讀同一word，unique requested bytes只有4，可能由一個sector供應再broadcast；不能把32次邏輯使用誤算成128個不同payload bytes。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Shared banks、broadcast 與 padding

採32個banks，每bank word4 bytes，bank=word index mod32。同一request內不同banks可並行；同bank不同word需多次服務。本模型對同一word的多lane read可broadcast，不把它當成不同word的衝突。

32×32 row-major array，column c=0的lane r讀index=32r，32個不同words都在bank0，形成32-way conflict。改成leading dimension33，index=33r，bank=r，分散到32個banks。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shared banks、broadcast 與 padding （續）

同一欄讀取：每格代表一個 lane 所選的 bank

stride 32：32 個不同 words 全落 bank 0

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

stride 33：lane r 映射到 bank r

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

假設 32 banks、每個 word 4 bytes；同 word 廣播不屬於此衝突。

Padding把shared用量由4096增至4224 bytes，可能使每SM可容納的blocks數跨過整數門檻。歷史Tesla用16-bank/half-warp服務規則，應以16個同時requests重算；不能把本節32-way結果直接標成該晶片實測。

A.6 浮點運算的次序也是模型

IEEE binary32有24-bit有效精度。取a=1+2^−23、b=1−2^−23、c=−1，三者可表示。精確ab=1−2^−46，若先捨入binary32 product，得到1，再加c得0。

若採一次捨入的IEEE FMA，先在精確乘加層計算ab+c=−2^−46，再捨入；此值可精確表示，所以結果是−2^−46，與兩次捨入不同。歷史G80 MAD不能無條件當成這種FMA，其subnormal、近似函數與例外支援也有版本限制。

Reduction改變加法樹也可能改變結果。例如binary32的(10^8+1)+(−10^8)得到0，而(10^8−10^8)+1得到1。平行演算法的整數正確性證明若依結合律，不能直接推出浮點bitwise相同；需另外指定誤差與可重現性要求。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A.7 GeForce 8800 的歷史結構

以 GeForce 8800 的歷史架構為例：8 個 TPC，每TPC兩個SM，每SM8個SP，因此8×2×8=128 SP；記憶體端有6個64-bit DRAM partitions，合計384-bit介面。SM另有shared memory、constant cache及SFU；texture與ROP負責不同資料路徑。

這個乘積是該圖的結構核對，不是今日GPU效能預測。不同世代可改變cache、warp排程、FP支援及memory transaction規則；core數相同也可能因clock、instruction mix與資料流不同而有不同throughput。SFU與ALU的peak只有在規格允許相應並行且workload真的使用時才可共同計數，不能把所有峰值直接相加。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.8 CSR SpMV 的矩陣與 arrays

取

$$A = \begin{bmatrix} 2 & 0 & -1 & 0 \\ 0 & 3 & 0 & 4 \\ 5 & 0 & 0 & 6 \end{bmatrix}, \qquad x = (1, 2, 3, 4)^T.$$

CSR使用Ap=[0,2,4,6]、Aj=[0,2,1,3,0,3]、Av=[2,−1,3,4,5,6]。Row i的非零項位於[Ap[i],Ap[i+1])，所以y0=2・1−1・3=−1，y1=3・2+4・4=22，y2=5・1+6・4=29。

Ap長度rows+1，最後值等於nnz；Aj皆在[0,columns)。矩陣可為非方形，不能把rows當作x的合法長度。本例x長4，輸出y長3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Row-per-thread 的正確性與不均衡

```
row = block_id * B + thread_id
if row < rows:
    total = 0
    for k = Ap[row] .. Ap[row+1]-1:
        total += Av[k] * x[Aj[k]]
    y[row] = total
```

每個row只有一個writer，loop invariant是total等於本row已處理非零項的乘積和；讀完區間即得該row點積。Empty row的區間長0，輸出0。

若同warp的31個rows各1項，另一row有64項，依masked loop的簡化模型要64輪，active lane-iterations=31+64=95，capacity=32×64=2048，利用率約4.64%。Warp-per-row可把長row的entries分給lanes再reduction，但短row可能浪費更多lanes；shared cache x也只在重用超過載入/同步成本時才有利。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inclusive 與 exclusive scan

對a=[3,1,7,0,4,1,6,3]，inclusive scan包含當前元素，結果[3,4,11,11,15,16,22,25]；exclusive scan只含之前元素，結果[0,3,4,11,11,15,16,22]。

Hillis–Steele每輪從上一輪snapshot讀取i−d，d=1,2,4。實際狀態為

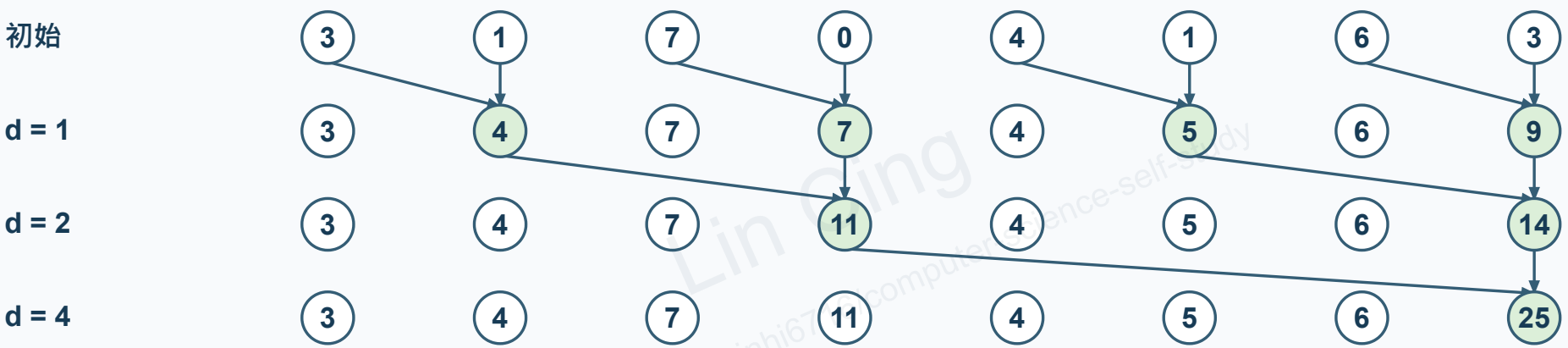
d	輪結束的array
初始	[3,1,7,0,4,1,6,3]
1	[3,4,8,7,4,5,7,9]
2	[3,4,11,11,12,12,11,14]
4	[3,4,11,11,15,16,22,25]

不使用snapshot或正確同步就直接原地改寫，可能讀到本輪已更新的值。此例加法數7+6+4=17；一般work Θ(n log n)，span Θ(log n)，不能只因depth小就稱work-efficient。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Work-efficient scan 的 up-sweep

Blelloch scan在完全二元樹先建立各區間總和。對n=8，d=1時把每對左值加到右端；d=2時合併兩對；d=4時得到整體總和。



綠色為本輪更新的右端點，其餘位置保持原值；root=25是全部八個輸入的和。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Work-efficient scan 的 up-sweep （續）

合併距離d	更新後array
1	[3,4,7,7,4,5,6,9]
2	[3,4,7,11,4,5,6,14]
4	[3,4,7,11,4,5,6,25]

每個internal node恰算兩個互不相交子區間之和，依樹高歸納得到該node的區間總和；共n−1次加法，depth log2 n。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Down-sweep 傳遞區間之前的總和

將root總和25另存，再把root設0。若parent持有其區間之前的prefix p，left subtree總和為t，則left child獲得p，right child獲得p+t。這兩個prefix分別恰為左右子區間之前的總和。

步驟	array
root設0	[3,4,7,11,4,5,6,0]
d=4	[3,4,7,0,4,5,6,11]
d=2	[3,0,7,4,4,11,6,16]
d=1	[0,3,4,11,11,15,16,22]

沿樹向下歸納，leaf i取得全部先前元素的和，故輸出exclusive scan。Down-sweep每internal node一次加法，共n−1；總work2(n−1)、span2log2 n，另計swap與root初始化。對非2次方n，以identity 0補到下一個2次方；n=0直接回傳空輸出。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

可重建的 shared-memory scan 偽碼

以下n為2次方，所有參與threads執行相同輪數；每輪互不重疊的k可平行。

```
for d = 1, 2, 4, ..., n/2:
    parallel for k = 0, 2*d, 4*d, ...:
        x[k+2*d-1] += x[k+d-1]
    block_barrier()
if thread_id == 0:
    total = x[n-1]
    x[n-1] = 0
block_barrier()
for d = n/2, ..., 2, 1:
    parallel for k = 0, 2*d, 4*d, ...:
        t = x[k+d-1]
        p = x[k+2*d-1]
        x[k+d-1] = p
        x[k+2*d-1] = p + t
    block_barrier()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

`total`只需一個指定thread保存，再依需要發布。這個原地scan不需要double buffer；Hillis–Steele的snapshot需求與此演算法不同，不能混用其同步規則。

多 blocks 的 scan 與全域邊界

N=10、每block4個數，輸入[3,1,7,0,4,1,6,3,2,5]。各block補0做local exclusive scan，block sums為[11,14,7]；對block sums再做exclusive scan得offsets[0,11,25]。

各local結果加自己的offset後，完整輸出為[0,3,4,11,11,15,16,22,25,27]，總和32。Phases依序為local scan與sums、scan sums、加offset；sums很多時可遞迴分層。Kernel邊界建立跨block完成順序，不要求所有blocks同時resident。

固定block size下，各層資料量形成幾何級數，所以總work O(N)；抽象平行模型的tree span為O(log N)，實際時間還含每層launch、memory traffic與有限SM排程。

Lin Ching
https://github.com/xinhi6716/computer-science-self-study

Reduction 的不變量與數值

同一個8元素array，可用stride4、2、1，把後半加到前半。每輪前半各位置代表互不重疊子集合的和，合併保持此性質，最後x0為全部總和25。

stride	本輪有效partial sums
4	[3+4,1+1,7+6,0+3]=[7,2,13,3]
2	[7+13,2+3]=[20,5]
1	[20+5]=[25]

總加法4+2+1=7。多block可先輸出block sums再reduction，或用規格支援的atomic操作合併。Atomic保證單次更新不互相遺失，不保證不同blocks的浮點相加順序固定。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Stable radix partition 的每個目的位址

採3-bit unsigned keys [7,2,5,0,3,4,1,6]。處理bit0時，定義z_i=1表示該bit為0；z=[0,1,0,1,0,1,0,1]，exclusive zero-prefix Z_i=[0,0,1,1,2,2,3,3]，總zeros Z=4。

Zero的目的位置是Z_i；one的目的位置是Z+(i−Z_i)。因此dest=[4,0,5,1,6,2,7,3]，scatter後為[2,0,4,6,7,5,3,1]。兩區內prefix都嚴格按原相對順序增加，所以partition stable、目的位置不重複，覆蓋0–7。

逐欄對照輸入位置、zero flag、先前 zero 數與目的位置：

i	0	1	2	3	4	5	6	7
key	7	2	5	0	3	4	1	6
z_i	0	1	0	1	0	1	0	1
Z_i	0	0	1	1	2	2	3	3
dest	4	0	5	1	6	2	7	3

Lin Cing https://github.com/xinhi6716/computer-science-self-study

LSD radix sort 的歸納與完整結果

在前一輪stable結果上處理下一個bit，實際三輪為

已處理bits	結果
bit0	[2,0,4,6,7,5,3,1]
bit0–1	[0,4,5,1,2,6,7,3]
bit0–2	[0,1,2,3,4,5,6,7]

歸納假設前b個低bits已排序。新partition按bit b分區，同區保留前一輪低bits順序，因此前b+1 bits有序。k-bit keys共k輪，每輪scan與scatter工作O(N)，總O(kN)，需可避免讀寫互相覆蓋的輸入/輸出storage。

這裡明定unsigned keys；signed或floating-point keys須先定義可保持所需順序的bit transform，不能直接把原始encoding當作相同unsigned順序。

N-body 的 force 與 acceleration

對target i，softened acceleration為

$$\mathbf{a}_i = G \sum_{j \neq i} \frac{m_j (\mathbf{x}_j - \mathbf{x}_i)}{(\|\mathbf{x}_j - \mathbf{x}_i\|^2 + \varepsilon^2)^{3/2}}.$$

目標粒子的質量不在加速度公式的分子中；若要求力，還要乘上該粒子的質量，即 $\mathbf{F}_i = m_i \mathbf{a}_i$。軟化參數 ε 可避免極近距離的奇異行為，但也改變了物理模型。

在一維例子中，四個位置依序為 [0,1,3,4]，質量和 G 都是 1；令 ε=0，並略過粒子對自己的作用。計算得到 $a_0 = 1 + 1/9 + 1/16 = 169/144$，$a_1 = -1 + 1/4 + 1/9 = -23/36$，另外 $a_2 = 23/36$、$a_3 = -169/144$。各粒子質量相同，因此總力為 0；這也能檢查成對作用的方向。有限精度及不同累加順序仍可能造成小誤差。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

N-body tile 的資料重用與兩道 barrier

每個 block 有 B 個目標執行緒，合作把 B 個來源粒子載入 shared memory，再由每個有效的目標執行緒累加該區塊的作用。以下偽碼中的 body 包含位置與質量。

```
i = block_id * B + thread_id
my = (i < N) ? body[i] : zero_body
acc = 0
for base = 0; base < N; base += B:
    j = base + thread_id
    sh[thread_id] = (j < N) ? body[j] : zero_body
    block_barrier()
    if i < N:
        for k = 0 .. min(B,N-base)-1:
            if base+k != i:
                acc += acceleration(my, sh[k])
    block_barrier()
if i < N:
    output[i] = acc
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

N-body tile 的資料重用與兩道 barrier （續）

第一道 barrier 確保所有來源資料載入完成；第二道防止較快的執行緒在其他人讀完之前覆寫資料。超出範圍的目標執行緒仍協助載入並參與兩道同步。每個有效目標都恰好處理每個不同於自身的有效來源一次，計算項與直接雙迴圈相同，因此仍需 $\Theta(N^2)$ 對粒子的運算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tile 減少哪一層流量

明定N為B倍數、無cache重用，naive每個target另載入一次my，並讀N個source（包括self，讀入後才跳過其計算）。Source reads=N^2；tiled有N/B個blocks，每block讀N個sources，為N^2/B。

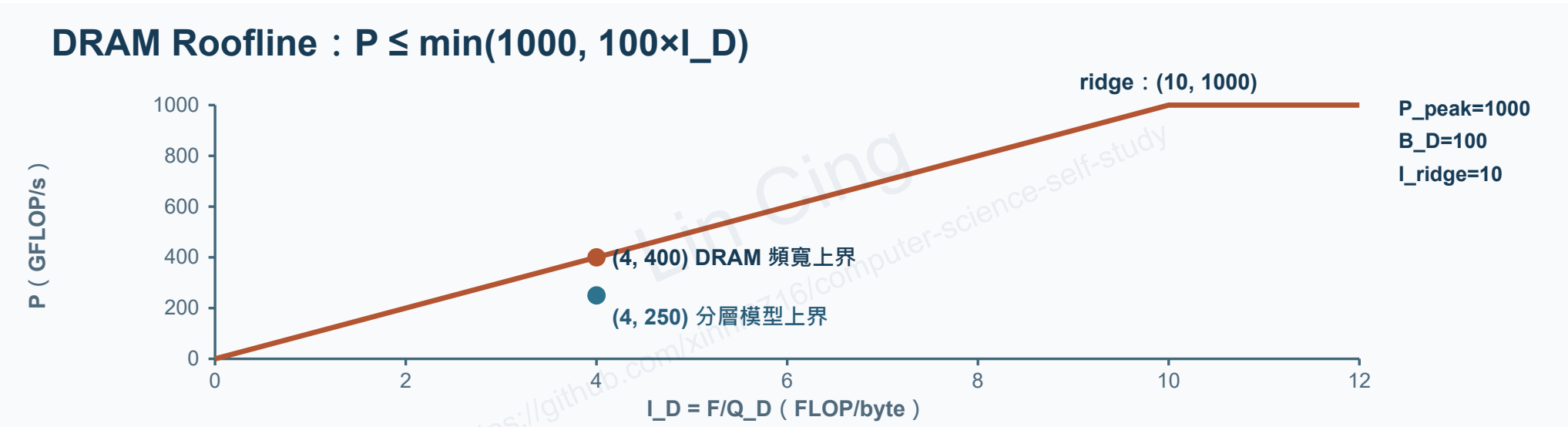
N=4、B=2時，source reads由16減到8；另加4個my loads，總position loads由20減到12，總流量只降為原本60%，不能宣稱所有流量恰減半。若compiler/cache重用self或其他sources，或body欄位寬度不同，需按實際介面bytes重算。

Shared內每個source仍可能被B個targets讀取，因此DRAM bytes降低不等於shared bytes也以相同比例降低。下一節以每個介面自己的Q計算Roofline限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分層流量的限制不能混乘

固定同一 phase $F = 10^9$ FLOPs、$P_C = 1000$ GFLOP/s。DRAM 實際讀寫 $Q_D = 0.25 \times 10^9$ bytes，持續頻寬 $B_D = 100$ GB/s；shared memory 實際讀寫 $Q_S = 4 \times 10^9$ bytes，持續頻寬 $B_S = 1000$ GB/s。這些是明定模型輸入，不是某張 GPU 的實測規格。



DRAM 層的橫座標是

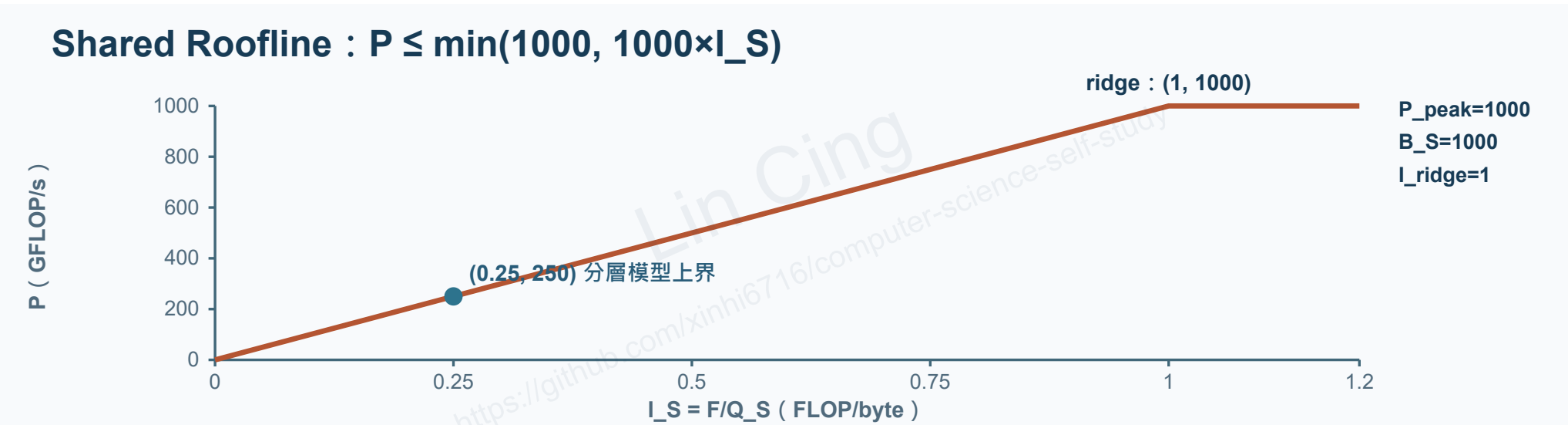
$$I_D = F/Q_D = 10^9/(0.25 \times 10^9) = 4\ \mathrm{FLOP/byte}.$$

分層流量的限制不能混乘 （續）

其 throughput 上界為

$$\min(P_C, I_D B_D) = \min(1000, 4 \cdot 100) = 400 \text{ GFLOP/s}.$$

DRAM roof 的折點是 $P_C/B_D = 10$；$(4, 400)$ 在斜線的 bandwidth 部分。



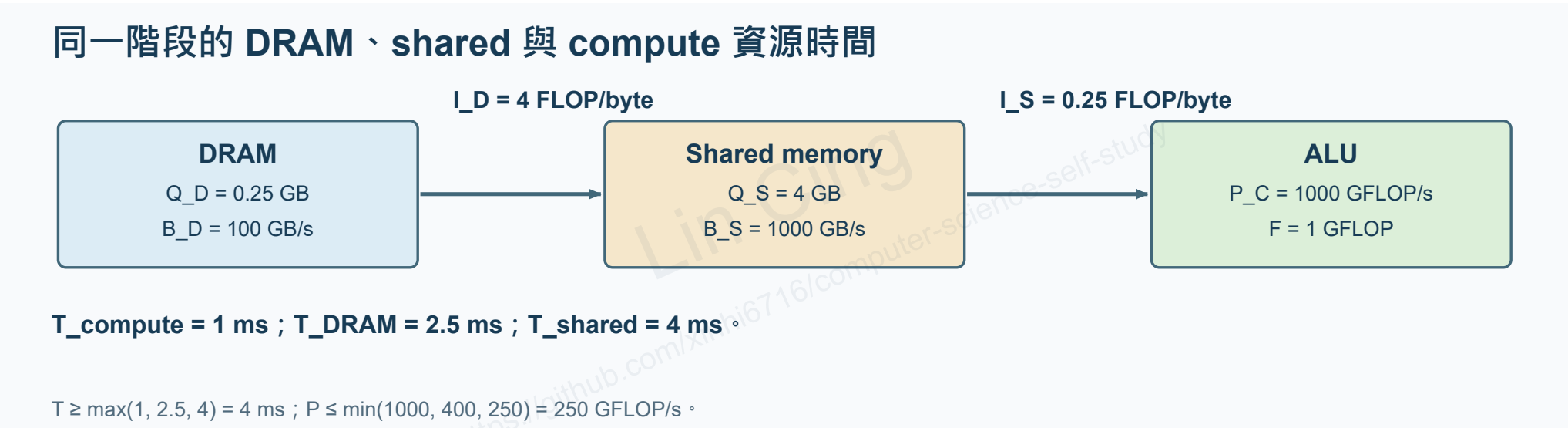
Shared 層要重新計算

$$I_S = F/Q_S = 10^9/(4 \times 10^9) = 0.25 \text{ FLOP/byte}.$$

分層流量的限制不能混乘 （續）

$$\min(P_C, I_S B_S) = \min(1000, 0.25 \cdot 1000) = 250 \text{ GFLOP/s}.$$

Shared roof 的折點是 1；1000 GFLOP/s 才是 plateau，$(0.25, 250)$ 位於斜線。250 不能畫成 shared roof 的水平上限。



三個 resource times 是

$$T_{\text{compute}} = 10^9/(1000 \times 10^9) = 0.001 \text{ s},$$

分層流量的限制不能混乘 （續）

$$T_D = 0.25 \times 10^9/(100 \times 10^9) = 0.0025\ \text{s}, \qquad T_S = 4 \times 10^9/(1000 \times 10^9) = 0.004\ \text{s}.$$

若各資源 traffic 可以重疊，

$$T \geq \max(1, 2.5, 4) = 4\ \text{ms},$$

$$P \leq F/T = 10^9/0.004 = 250\ \text{GFLOP/s}.$$

最終 $(I_D, I_S) = (4, 0.25)$ 對應的跨層上界是 250 GFLOP/s。它是 declared model bound，不是 measured throughput；實際 kernel 還可能因 bank replay、同步、occupancy 或額外 traffic 低於此值。

層級	bytes	強度	可持續頻寬	該層上限
compute	不適用	不適用	$P_C = 1000$ GFLOP/s	1000 GFLOP/s
DRAM	0.25×10^9	4 FLOP/byte	100 GB/s	400 GFLOP/s
shared	4×10^9	0.25 FLOP/byte	1000 GB/s	250 GFLOP/s
合併	各層獨立	各層獨立	各層獨立	250 GFLOP/s

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A.9 用明確反例檢查性能推論

高occupancy未必快，因為減register引起spill可增加memory traffic。連續access未必滿DRAM頻寬，因為outstanding requests、channel分配或依賴鏈仍可限制；shared memory也可能因bank conflicts與barriers而變慢。

單一kernel的peak不能當作應用speedup。相同FLOP數而含不同精度、近似函數、FMA計數或transfer邊界，也不可直接比較。若量測P超過由同phase之F/Q與有效頻寬得到的roof，先核對FLOP計數、cache/interface bytes、時間範圍及頻寬假設，而非立即宣稱推翻資源上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.10 正確性先於 mapping 選擇

CSR需保持每row完整點積與唯一writer；scan需保留prefix不變量與同步順序；radix需stable partition；N-body需覆蓋全部pair並正確管理tile生命期。以上契約成立後，才比較row-per-thread、warp-per-row、tile size或register使用量。

圖、trace與有限模型核對可定位錯誤，但一般性正確性分別依賴唯一索引、loop/tree invariant與歸納。GPU世代改變不會讓不正確的同步或越界程式因「通常同時執行」而變得合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.11 歷史架構與可遷移方法

GPU由固定graphics pipeline演進為unified processors，CUDA/PTX把大量thread程式映到可伸縮的硬體。Warp寬度、banks、cache與精度支援可變，仍需分清latency、throughput、resource capacity與資料交換的語意。

本章歷史GeForce 8800的結構來自原書，sector/bank/occupancy數例則明定為獨立抽象模型。把模型的參數與推導分開，讀者才可替換成新硬體的真規格重新分析，而不是背一組過時常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

Patterson與Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，Appendix A，A.1–A.11，印刷A-3–A-77（PDF實體732–806）。Warp排程與divergence見A-28–A-30（PDF757–759）；16-bank模型見A-40（PDF769）；GeForce結構FigA.7.1見A-47（PDF776）；scan/reduction/radix與N-body見A-60–A-69（PDF789–798）。

Williams、Waterman與Patterson，〈Roofline: An Insightful Visual Performance Model for Multicore Architectures〉，*Communications of the ACM*52(4)，2009，DOI。

分層bytes定義另見NERSC Roofline文件。本文的32-bank、32-byte sectors、occupancy及計算數據是各頁明定的教學模型，未冒充同一款GPU的實測。

參考資料與署名

David A. Patterson、John L. Hennessy，Computer Organization and Design，Revised Fourth Edition，附錄 A；計算機組織與結構（CSO）課程投影片。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study